ArabicMTDiagnostics at AlexandriaX-2026: Detecting and Classifying Errors in Dialectal Arabic Machine Translation Outputs

Sarah Yassine
Lebanese University / Beirut, Lebanon
sarah.yassine.2@st.ul.edu.lb

Abstract

Dialectal Arabic MT error detection asks a system to locate error spans in a machine translation and label each with a linguistic error category. It is hard because the dialects lack a standardised orthography and the error categories are severely imbalanced. In Subtask 3 of the AlexandriaX 2026 shared task, the input is an English sentence and its machine translation into one of five Arabic dialects, and the output is character-level error spans with their LQM categories. The central difficulty is collapse onto the majority *sociolinguistics* class, 58% of training spans, leaving rare categories undetected. To address this, we fine-tune NileChat-3B-Base with QLoRA on a single 8 GB consumer GPU, casting the task as schema-prompted JSON generation and combining character-position annotations that let the model copy offsets rather than count them, a category guide with one worked example per error type, and self-consistency decoding merging five samples by character-level union voting. Our system scores 0.3484 Overall on the official test set. We report two negative results: early stopping on validation loss degrades performance, since rare categories are acquired only in later epochs, and fine-tuning on the skewed distribution lowers macro-F1 below the un-fine-tuned baseline until the category guide is added.

1 Introduction

Machine translation error detection for dialectal Arabic is the task of locating the exact spans of a translated sentence that are wrong and identifying what kind of error each span represents. It is harder than the equivalent task for standardised languages: Arabic dialects have no fixed orthography, so a single word may be written in several acceptable ways, and a translation can be grammatically well formed yet still wrong because it uses the wrong variety or the wrong register for its context.

El Mekki et al. (2026b) introduce Alexandria, a multi-domain dialectal Arabic machine translation dataset, which forms the basis of the AlexandriaX 2026 shared task on dialectal Arabic machine translation (El Mekki et al., 2026a). We present our submission to Subtask 3 of that shared task. Given an English source sentence and its machine translation into one of five Arabic dialects, Egyptian, Emirati, Mauritanian, Moroccan, and Palestinian, the system must return the character offsets of every error span and assign each span one category from the LQM error taxonomy: *sociolinguistics*, *pragmatics*, *semantics*, *morphosyntax*, *orthography/writing conventions*, or *graphetics*.

The main challenge is the shape of the label distribution rather than the dialects themselves. In training, the five dialects are close to balanced, spanning 169 to 263 instances, but the error categories are not: *sociolinguistics* accounts for 58% of all annotated spans, while *morphosyntax*, *orthography* and *pragmatics* together account for under 18%. A model trained on this distribution learns to answer *sociolinguistics* whenever it is uncertain, and because the official score is macro-averaged over categories, each rare category carries the same weight as the majority one, making this collapse costly.

We address this with a single-stage generative system: NileChat-3B-Base (El Mekki et al., 2025) fine-tuned with QLoRA on one 8 GB consumer GPU, with the task cast as schema-prompted JSON generation. Our submission ranked 6th on the Subtask 3 leaderboard with an Overall score of 0.3484. We release our code, notebook, and experiment configuration publicly.[1]

Our contributions are: (1) a compact system for dialectal Arabic MT error detection that trains end to end on a single consumer GPU; (2) two prompt-

[1] https://github.com/SarahYassine/AlexandriaX_Subtask3_ArabicMTDiagnostics

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 827–833
October 28-29, 2026 ©2026 Association for Computational Linguistics

level techniques targeting the two halves of the task separately, character-position annotation for span localisation and a category guide for minority-class recovery; and (3) two negative results, that early stopping on validation loss degrades task performance, and that fine-tuning on the skewed label distribution reduces macro-F1 below the un-fine-tuned baseline until the guide is introduced.

2 Background

2.1 Task Setup

Subtask 3 evaluates machine translation output rather than producing it. Each instance provides a translation direction, an English source sentence, and a machine translation of that sentence into the corresponding Arabic dialect.

Input. Each instance is a triple of a direction tag such as `ENG_EGY`, an English source sentence, and its dialectal translation. We do not pass the translation as raw text. Instead we prefix every whitespace-delimited token with the character offsets it occupies, so that the positions the model must report are present in its input and can be copied rather than counted. A translation such as مانقدرش نقولك، يُمي، becomes:

`[0:7]` مانقدرش `[8:14]` نقولك، `[15:20]` يُمي،

Output. The system returns a JSON object with a single key whose value is the list of error spans, each carrying the erroneous text, its start and end offsets, and its category, for example `{"text":"...", "start":0, "end":7, "category":"semantics"}`. A translation may contain any number of spans, including none. Offsets are half-open character positions into the machine translation string, indexed in Unicode code points as Python indexes a `str`, so a span covers the characters from `start` up to but not including `end`.

2.2 Dataset

The shared task data comes from Alexandria (El Mekki et al., 2026b), with error annotations following the LQM taxonomy (Magdy et al., 2026). LQM organises translation errors into six linguistically grounded levels and provides 6,113 expert-annotated error spans over 3,495 erroneous sentences drawn from a bidirectional corpus of 3,850 sentences across seven Arabic varieties. Subtask 3 uses five of them, excluding Jordanian and Yemeni.

The subtask releases 1,125 training, 138 development, and 145 test instances. Across the 1,997 annotated training spans, *sociolinguistics* accounts for 1,157 (57.9%), *semantics* 485 (24.3%), *morphosyntax* 164 (8.2%), *orthography* 106 (5.3%), *pragmatics* 83 (4.2%), and *graphetics* 2 (0.1%). *Graphetics* never appears in the development gold annotations, so we cannot measure it and do not target it; two training spans are too few to support any claim about whether it is learnable in principle. This skew, rather than the dialect distribution, is what shapes system behaviour.

2.3 Track Participation

We participated in Subtask 3 only, submitting a single system to the official leaderboard, and did not take part in Subtask 1 or Subtask 2.

2.4 Related Work

Translation into Arabic dialects is itself an open problem: Baniata et al. (2021) apply a Transformer with subword units between Arabic vernaculars and MSA, noting that free word order and sparse dialectal vocabulary make rare-word handling persistently difficult. The systems Subtask 3 diagnoses inherit these difficulties. Span-level annotation of Arabic has a longer history than span-level MT evaluation: Schneider et al. (2013) tag Arabic text with supersenses at the span level, using MT to transfer English annotation resources. Closer to our task, Alhafni et al. (2023) show that Arabic grammatical error detection benefits substantially from pretrained Arabic models, though their errors are grammatical rather than sociolinguistic and their text is written natively. LQM (Magdy et al., 2026) extends this line by separating dialect- and culture-specific failures from surface-level ones, necessary in a diglossic setting where a translation can be grammatical yet use the wrong variety. We operationalise that taxonomy as a generation target.

3 System Overview

3.1 Design Rationale

The task has two halves that fail for different reasons: span localisation fails because a generative model asked to count characters in an Arabic string does so unreliably, and classification fails because the label distribution is skewed. We keep a single model and a single generation pass, addressing

each half with its own intervention in the prompt and a third at decoding time (Section 4).

3.2 Model Choice

We build on NileChat-3B-Base (El Mekki et al., 2025), a 3B-parameter model derived from Qwen2.5-3B through continued pretraining on Arabic dialectal text, released as `UBC-NLP/NileChat-3B-Base`.[2] We chose it because it is dialect-aware rather than MSA-only, and because at 3B parameters it fits on an 8 GB consumer GPU under 4-bit quantisation.

One caveat is worth stating plainly. NileChat's continued pretraining targets Egyptian and Moroccan specifically, so three of our five dialects, Mauritanian, Palestinian and Emirati, fall outside its pretraining focus and are supported only by the 1,125 fine-tuning instances. Section 5 reports what this does, and does not, predict about per-dialect performance. Beyond this pretrained model we use no external resources: the worked examples in our category guide are drawn from the provided training split.

3.3 Prompt Design

Each instance is rendered as one instruction prompt with four parts: a task instruction, the category guide, the instance fields, and the required output schema. The model continues this prompt with a JSON object.

Character-position annotation. Rather than asking the model to count characters, we compute each token's offsets and place them in the input, so the offsets the model must report can be copied. This is the single change that most improved exact-span agreement (Section 5).

Category guide. To counter the collapse onto the majority label, the prompt carries a compact guide listing all five learnable categories, each with a one-line definition and one worked example taken from the training split. The guide is identical for every instance and costs roughly 212 tokens. Its purpose is to give the rare categories an explicit description at inference time, since they are too infrequent to be learned reliably from examples alone. The full guide is reproduced in Appendix A.

[2] `https://huggingface.co/UBC-NLP/NileChat-3B-Base`. We loaded the default `main` revision throughout; we did not pin a commit hash.

3.4 Fine-Tuning Strategy

We fine-tune with QLoRA. The base model is loaded in 4-bit NF4 with double quantisation and `bfloat16` compute, reducing the frozen weights to roughly 2 GB and leaving room for optimiser state and activations within 8 GB. All learning is carried by LoRA adapters with rank-stabilised scaling applied to all linear layers, training 29.9M parameters, about 1% of the base model. Training uses completion-only loss, so the model is scored on the JSON answer rather than on the long prompt.

3.5 Submitted System

We submitted a single system: NileChat-3B-Base fine-tuned with QLoRA on the position-annotated prompt with the category guide, decoded with five-sample self-consistency at vote threshold one and a fixed seed. We also developed a two-stage variant separating span detection from classification; it was not submitted, because it did not improve the development Overall score.

4 Experimental Setup

4.1 Data Splits and Preprocessing

We use the official splits without modification. The test split is released without gold annotations, so all development-time comparisons use the 138 development instances and only the submitted run is scored on test. We held out no separate tuning split, so development figures are optimistically biased.

Preprocessing is deliberately minimal. We apply **no** normalisation to the Arabic text: no diacritic stripping, no alef or ta' marbuta unification, no punctuation folding. This departs from common Arabic NLP practice for two task-specific reasons. First, *orthography/writing conventions* is one of the categories the system must detect, so normalising spelling variation would erase the evidence for the errors being predicted. Second, gold spans are character offsets into the raw translation string, and normalisation would shift those offsets and invalidate the annotation. The only transformation is the position annotation of Section 2, computed by segmenting on whitespace. Sequences longer than 768 tokens are truncated, affecting 5.8% of training instances; we chose 768 as the smallest budget accommodating the category guide.

4.2 Training Configuration

We train for 10 epochs with an effective batch size of 8 (batch size 1, gradient accumulation 8), a learning rate of 2×10^{-4} under a cosine schedule with warmup ratio 0.03, LoRA rank 16 with $\alpha = 32$ and dropout 0.05, and the paged AdamW 8-bit optimiser with gradient checkpointing. A run takes approximately 15 hours on our hardware. Appendix B lists the full configuration.

4.3 Decoding and Post-processing

At inference we replace greedy decoding with self-consistency. For each instance we sample five generations with temperature 0.7, nucleus sampling at $p = 0.9$, and 256 new tokens, then merge them with a character-level vote: every character accumulates one vote from each generation whose spans cover it, contiguous runs reaching the vote threshold become the final spans, and each span's category is decided by an overlap-weighted vote among the sampled spans touching it. We use a vote threshold of one, the union of all five samples (Section 5).

Sampling is seeded per instance, using a fixed base seed plus the instance index, so any subset reproduces identically; unseeded reruns of an identical configuration varied by roughly 0.02 Overall.

Decoding is unconstrained: we prompt for the output schema and parse whatever the model produces, with no grammar, logit masking, or constrained-decoding library. Post-processing is limited to parsing and does not repair generations. A completion that fails to parse as JSON yields an empty span list for that sample, as does an item that is not an object or that omits either offset. Offsets are clipped to $[0, |\text{MT}|)$ and reversed spans, where `start` $\geq$ `end`, contribute no characters, so both are dropped in effect. Duplicate and overlapping spans need no special handling: union voting merges them into contiguous character runs, so the emitted spans are disjoint by construction, and each run's `text` field is re-derived by slicing the translation, which makes a text/offset mismatch impossible in our output. A merged run takes the category with the largest overlap-weighted vote among the sample spans touching it; a category string the model invents is passed through unchanged and scored as a miss.

Across the 145 test instances, every row received at least one predicted span, so no test prediction was lost to a parse failure. At the level of individual samples, 13 of the 690 sample slots on the development set (1.9%) were empty. That figure is an upper bound on the parse-failure rate, since an empty list is also the correct answer for a translation with no errors.

4.4 Tools and Libraries

Experiments were run locally on a single NVIDIA RTX 5070 Laptop GPU with 8 GB of VRAM, under Python 3.11.15 with PyTorch v2.9.1+cu128 and CUDA 12.8. We used Transformers v5.13.0, Datasets v5.0.0, PEFT v0.19.1, TRL v1.7.1, Accelerate v1.14.0, bitsandbytes v0.49.2, and NumPy v2.4.6. Project pages for all libraries are listed in Appendix C, and full scripts, the training notebook, and per-run configurations are available in our public repository.

4.5 Evaluation Metrics

The official evaluation reports four quantities. **Overlap F1** compares predicted and gold character masks directly, so partially correct spans receive partial credit. **Exact Match F1** counts a span only when both offsets match exactly. **Error Class Macro-F1** is the unweighted mean of per-category F1 over overlap-matched span pairs, so each category contributes equally regardless of frequency. The **Overall** score is the mean of Overlap F1 and Error Class Macro-F1.[3]

5 Results

5.1 Official Results

Table 1 shows the final leaderboard. We placed 6th of six teams, with an Overall score of 0.3484, an Exact Match F1 of 0.1222, an overlap F1 of 0.3818, and an error-class macro-F1 of 0.3151. Both components of the Overall score are low relative to the field: our overlap F1 is the lowest of the six, and our macro-F1 is fifth, exceeding only Axiom's 0.3040. Span detection is the weaker of the two, and the analysis below concentrates on it, but classification is also near the bottom of the field, so we do not treat span detection as the sole constraint on our score.

Per-dialect, Emirati is strongest (overlap F1 0.4409, macro-F1 0.3899), then Palestinian (0.4269, 0.3036), Moroccan (0.3487, 0.2149), Mauritanian (0.3375, 0.3434) and Egyptian

[3] We verified this against the official scoring output for our submission: the reported Overall score is reproduced exactly by averaging Overlap F1 and Error Class Macro-F1.

Team	Overall	EM	Ovl.	Macro
Thakaa	0.4982	0.2319	0.5426	0.4538
Namaa Comm.	0.4641	0.2239	0.5448	0.3834
TTlab	0.4091	0.1940	0.4600	0.3582
REGLAT	0.4021	0.0041	0.4272	0.3770
Axiom	0.3926	0.1664	0.4813	0.3040
ArabicMTDiagnostics	0.3484	0.1222	0.3818	0.3151

Table 1: Official Subtask 3 leaderboard, in final ranking order. EM = Exact Match F1, Ovl. = Overlap F1, Macro = Error Class Macro-F1.

(0.3298, 0.3053). The dominant pattern is systematic rather than dialectal: **overlap precision is roughly half of recall in every dialect**, 0.24–0.33 against 0.46–0.65; for Egyptian, 7 true positives against 73 false positives. The system over-predicts heavily, the expected behaviour of union voting and our clearest target for future work. Pretraining coverage does not predict performance: NileChat is continued-pretrained on Egyptian and Moroccan, yet these are the two weakest dialects by macro-F1.

5.2 Component Analysis

Development figures below use our re-implementation of the official metric and are not directly comparable to test scores. Position annotations raise exact-span F1 from 0.0475 to 0.1371, but the same step lowers macro-F1 from 0.3815 to 0.2748, below the un-fine-tuned baseline: fine-tuning on the skewed distribution teaches the model to answer *sociolinguistics* more often, and under macro-averaging that trade is negative. The category guide reverses this, lifting macro-F1 to 0.4198 and giving our best Overall, 0.3975; *morphosyntax* rises from 0.250 to 0.588 F1 and *orthography* from 0.000 to 0.250, while *sociolinguistics* is unchanged. Prompt-level description of the rare categories, rather than more training on them, is what made fine-tuning worthwhile. Stricter self-consistency voting degraded Overall monotonically, from 0.3652 at union to 0.2877 at unanimity, so we kept the union.

Pragmatics is never predicted correctly, scoring 0.000 in every configuration including the un-fine-tuned baseline; every pragmatics span overlapping a prediction is labelled *sociolinguistics* or *semantics*. The two are adjacent in the LQM hierarchy (Magdy et al., 2026), and with 83 training spans the distinction is not learnable by prompt description alone. Because *graphetics* never appears in the development gold annotations, macro-averaging there runs over the five remaining categories, so a category stuck at 0.000 forgoes at most $1/5 = 0.2$ of macro-F1 and hence at most 0.1 of Overall. That is an upper bound, reached only if pragmatics were predicted perfectly; it exceeds the combined gain of every successful component, which is why we single it out.

All figures other than the leaderboard and per-dialect scores are development-set results obtained before the deadline; we made no post-submission modifications.

6 Conclusion

We presented our submission to Subtask 3 of the AlexandriaX 2026 shared task, fine-tuning NileChat-3B-Base with QLoRA on a single 8 GB consumer GPU and combining character-position annotations, a compact category guide, and self-consistency decoding. It achieved an Overall score of 0.3484 on the official test set, ranking 6th.

Two lessons emerge. Span precision, roughly half of recall in every dialect, is the largest single drag on our score, and the union voting that maximises coverage is the direct cause; classification is also weak relative to the field, so the two are best read together. More transferable, the severity of the category imbalance means standard practice can actively mislead: fine-tuning lowered macro-F1 below the un-fine-tuned baseline, and early stopping on validation loss lowered it further. On imbalanced span-labelling tasks, a prompt-level description of the rare classes proved more effective than either more training or earlier stopping.

Limitations

Our development set contains 138 instances, and per-dialect subsets are 21–34 instances, so single-run differences of ± 0.02 Overall are within sampling noise. We fixed the decoding seed to make comparisons reproducible, but did not average over multiple training seeds, and all reported configurations are single-run. We also did not hold out a separate tuning split: the development set served both for model selection and for error analysis, so development figures are optimistically biased.

Because test labels are not released, all development figures use our own re-implementation of the official metric. It reproduces the official Overall formula, but we could not verify it matches the organisers' scorer in every detail, so development and test numbers should not be compared directly.

Every training run takes roughly 15 hours on one 8 GB GPU, which limited hyperparameter exploration: LoRA rank, learning rate, and schedule were never swept, and all results use the first configuration that trained successfully. *Graphetics* is unmodelled, with two training spans and none in development gold. Finally, our system was trained and evaluated only on the five dialects of Subtask 3, and we make no claim about the remaining Arabic varieties in LQM.

Two configurations we tested did not improve on the submitted system and are reported in Appendix D: early stopping on validation loss, and a two-stage pipeline separating span detection from classification.

Acknowledgments

We thank the AlexandriaX 2026 shared task organisers for providing the dataset, the evaluation infrastructure, and the LQM annotation framework.

References

Bashar Alhafni, Go Inoue, Christian Khairallah, and Nizar Habash. 2023. Advancements in Arabic grammatical error detection and correction: An empirical investigation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6430–6448, Singapore. Association for Computational Linguistics.

Laith H. Baniata, Isaac K. E. Ampomah, and Seyoung Park. 2021. A transformer-based neural machine translation model for Arabic dialects that utilizes subword units. *Sensors*, 21(19):6509.

Abdellah El Mekki, Houdaifa Atou, Omer Nacar, Shady Shehata, and Muhammad Abdul-Mageed. 2025. NileChat: Towards linguistically diverse and culturally aware LLMs for local communities. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10967–10991, Suzhou, China. Association for Computational Linguistics.

Abdellah El Mekki, AbdelRahim A. Elmadany, Samar M. Magdy, Saad Ezzini, Mo El-Haj, Mustafa Jarrar, Zaid Alyafeai, Bernard Ghanem, and Muhammad Abdul-Mageed. 2026a. AlexandriaX 2026: The First Shared Task on Dialectal Arabic Machine Translation. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Abdellah El Mekki, Samar M. Magdy, Houdaifa Atou, Ruwa AbuHweidi, Baraah Qawasmeh, Omer Nacar, Thikra Al-hibiri, Razan Saadie, Hamzah A. Alsayadi, Nadia Ghezaiel Hammouda, Alshima Mohammed Alkhazimi, Aya Hamod, Al-Yas Yaqoob Al-Ghafri, Wesam El-Sayed, Asila Ismail al Sharji, Mohamad Ballout, Anas Belfathi, Karim Ghaddar, Serry Sibaee, and 28 others. 2026b. Alexandria: A multi-domain dialectal Arabic machine translation dataset for culturally inclusive and linguistically diverse LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32567–32592, San Diego, California, United States. Association for Computational Linguistics.

Samar M. Magdy, Fakhraddin Alwajih, Abdellah El Mekki, Wesam El Sayed, and Muhammad Abdul-Mageed. 2026. LQM: Linguistically motivated multidimensional quality metrics for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 40470–40493, San Diego, California, United States. Association for Computational Linguistics.

Nathan Schneider, Behrang Mohit, Chris Dyer, Kemal Oflazer, and Noah A. Smith. 2013. Supersense tagging for Arabic: the MT-in-the-middle attack. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 661–667, Atlanta, Georgia. Association for Computational Linguistics.

A Category Guide

The category guide inserted into every prompt is reproduced below. Each entry gives a one-line definition and one worked example drawn from the training split.

> **sociolinguistics**: wrong dialect, formality or register. E.g. نعم is formal MSA, not dialectal.
> **pragmatics**: wrong idiom, tone or implied meaning. E.g. محروق is the wrong figurative idiom for "brokenhearted".
> **semantics**: wrong literal meaning or word sense. E.g. بفكر ("I think") is wrong for "I feel like".
> **morphosyntax**: wrong grammar (agreement, tense, voice, order). E.g. بيعوا is active third person, not passive "you were sold".
> **orthography_writing_conventions**: wrong spelling or punctuation. E.g. ماش is a nonstandard spelling.

B Full Training Configuration

Hyperparameter	Value
Base model	NileChat-3B-Base
Quantisation	4-bit NF4, double quant.
Compute dtype	bfloat16
LoRA rank r	16
LoRA α	32
LoRA dropout	0.05
Target modules	all linear
rsLoRA	enabled
Trainable parameters	29.9M ($\approx$1%)
Epochs	10
Batch size	1
Gradient accumulation	8
Learning rate	2×10^{-4}
LR schedule	cosine
Warmup ratio	0.03
Max sequence length	768
Optimiser	paged AdamW 8-bit
Gradient checkpointing	enabled
Loss	completion-only
Random seed	42

Table 2: Full training configuration for the submitted system. The base model is `UBC-NLP/NileChat-3B-Base` on the Hugging Face Hub.

C Tools and Libraries

Project pages for the libraries used in this work:

- PyTorch v2.9.1+cu128 — https://pytorch.org/
- Transformers v5.13.0 — https://huggingface.co/docs/transformers/index
- Datasets v5.0.0 — https://huggingface.co/docs/datasets/index
- PEFT v0.19.1 — https://huggingface.co/docs/peft/index
- TRL v1.7.1 — https://huggingface.co/docs/trl/index
- Accelerate v1.14.0 — https://huggingface.co/docs/accelerate/index
- bitsandbytes v0.49.2 — https://github.com/bitsandbytes-foundation/bitsandbytes
- NumPy v2.4.6 — https://numpy.org/

D Negative Results

Early stopping on validation loss. Validation loss reaches its minimum at epoch 2.84 (0.2948) and rises steadily to 0.4859 by epoch 10, while training loss falls to 0.018: by the usual reading, clear overfitting. Restoring the best-validation-loss checkpoint instead lowers Overall from 0.3975 to 0.2775. The per-category breakdown explains why: at that checkpoint *morphosyntax* and *orthography* both score 0.000, the two categories the guide had repaired. Rare categories are acquired only in the later epochs that validation loss labels as overfitting. Token-level loss is dominated by frequent structure, the JSON scaffolding and the majority label, and is a poor proxy for span and category F1 under this degree of imbalance.

Two-stage pipeline. Separating span detection and classification into two adapters, with dialect-balanced and category-balanced oversampling respectively, produces the best overlap F1 we obtained (0.4052) but a lower macro-F1 than the submitted system, and a lower Overall (0.3842 against 0.3975). Category-balanced oversampling did not deliver its intended effect: evaluated on gold spans, so that span-detection error is excluded, *morphosyntax* recall fell from 0.50 to 0.14 despite fourfold oversampling, while *orthography* rose from 0.17 to 0.36. Duplicating a small pool of unique examples appears to encourage memorisation of their surface forms rather than generalisation.